

MLOps in Action: A Production- Ready Titanic Survivability Pipeline

Capstone Project

Presentation - TEAM 4

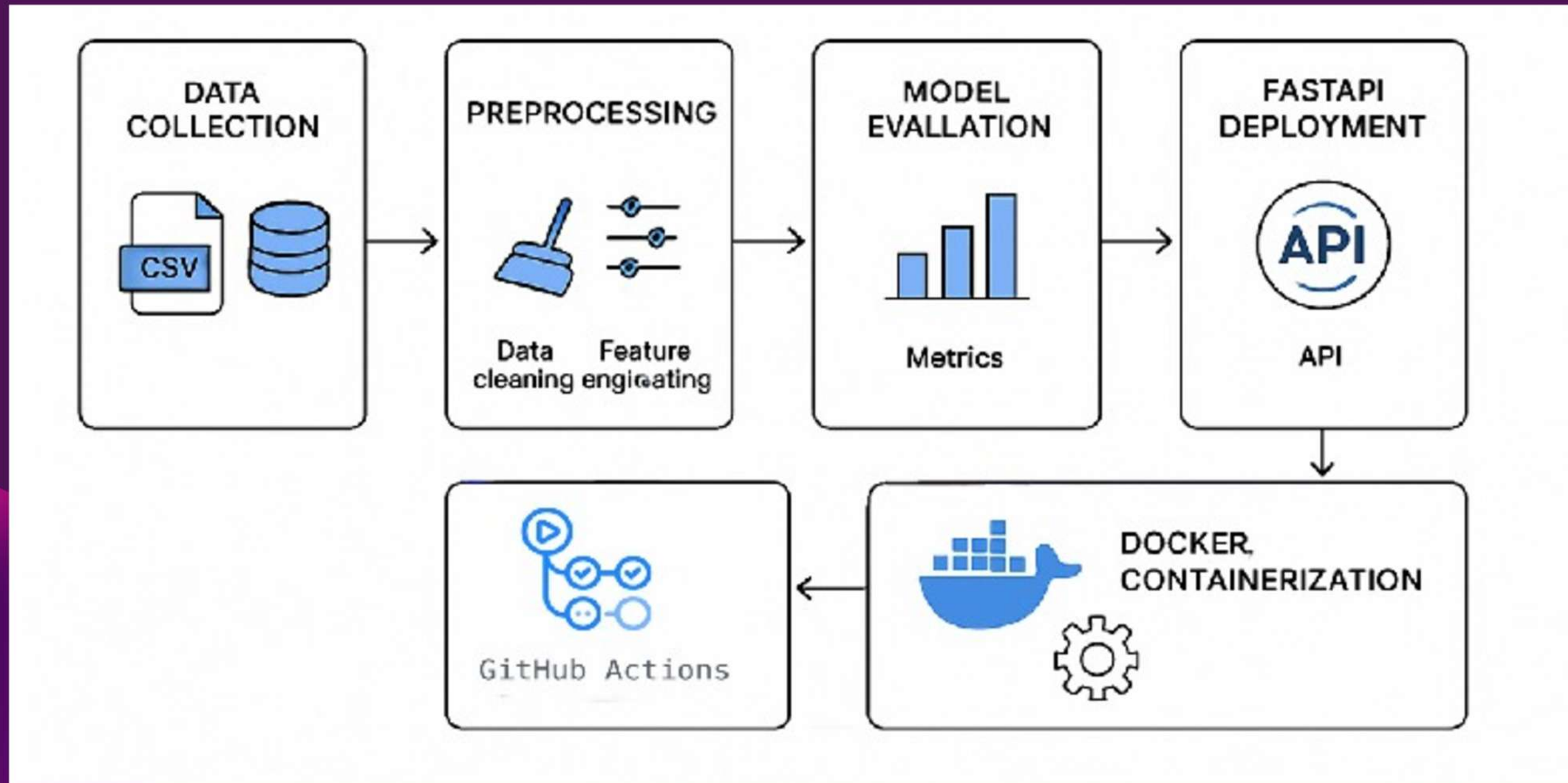
The Problem: Beyond the Notebook

The Goal: Predict passenger survival on the Titanic.

The Real-World Challenge: A model in a Jupyter Notebook is not a product. Moving it to a reliable, automated production environment is complex.

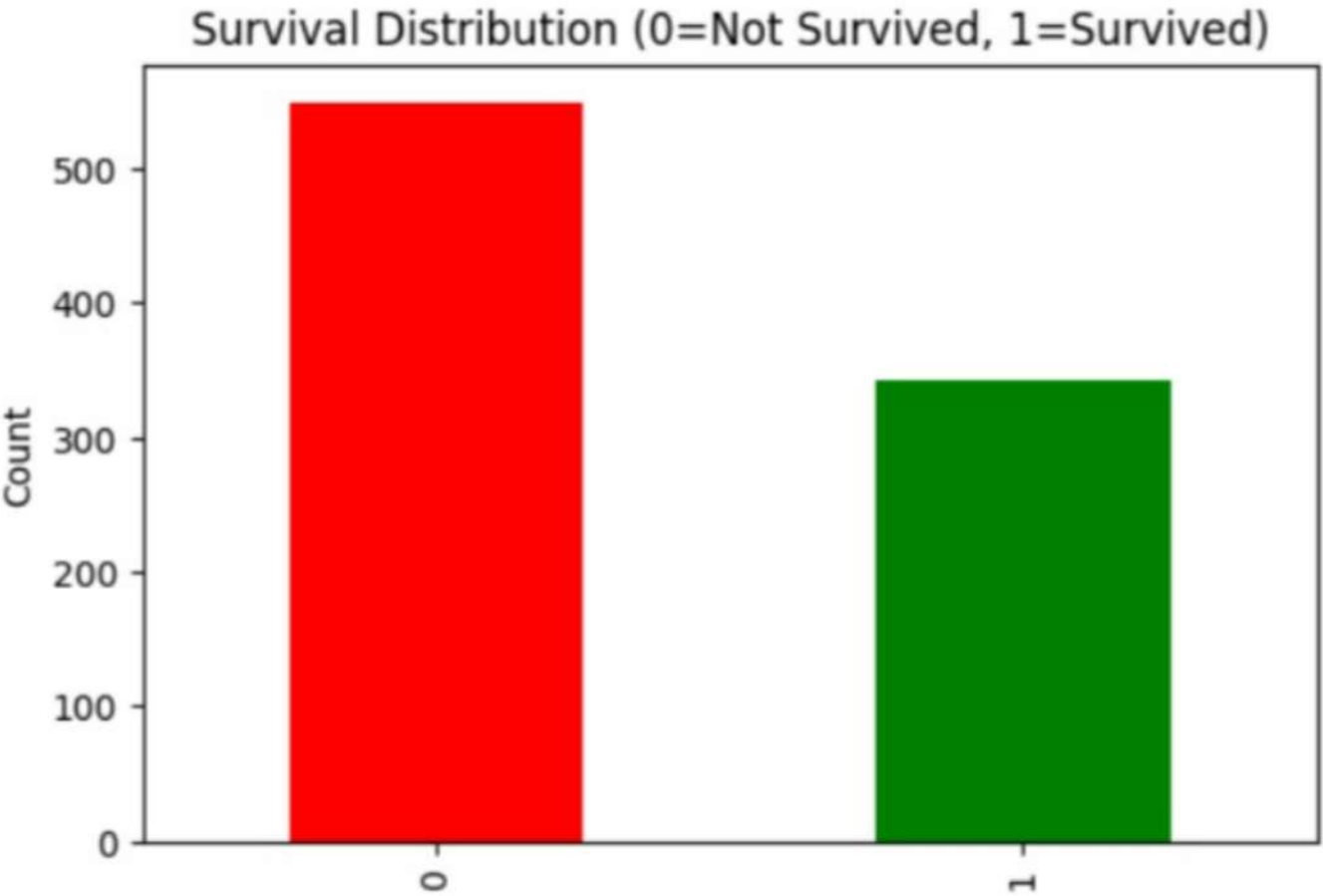
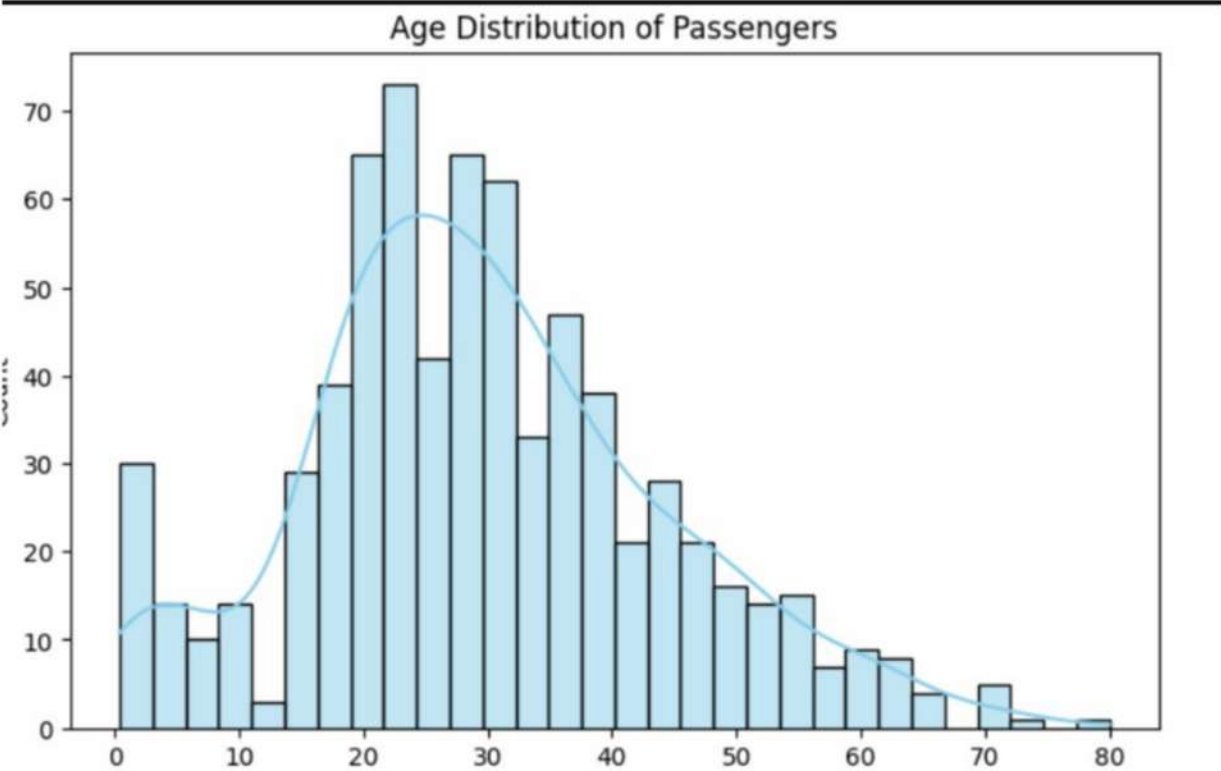
- How do we ensure results are reproducible?
- How do we automate testing and deployment?
- How do we serve the model reliably at scale?
- How do we monitor its performance and health over time?

Solution Architecture



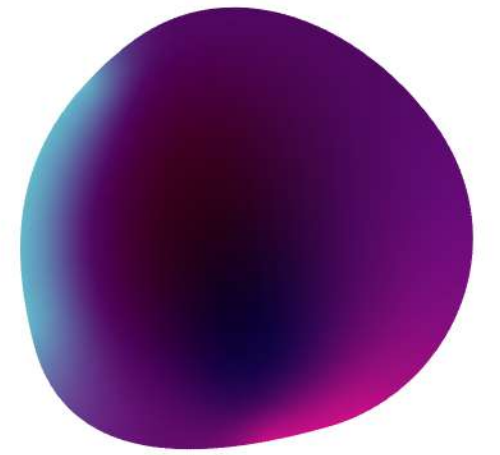
About Dataset

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S



Step 1: Versioning & Reproducibility

- **Git & GitHub:** Manages every version of our source code (src/, api/, tests/). Every change is tracked.
 - Treats our data (train.csv) and models (model.pkl) like code without bloating the Git repository.
 - Ensures that checking out a specific commit gives you the exact version of the data used to train that model.



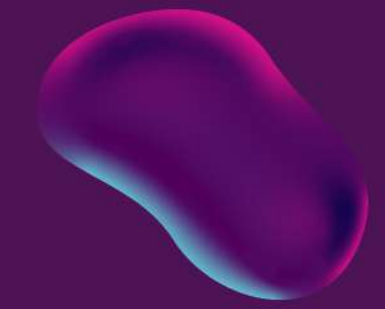
Step 2: Experiment Tracking & Model Selection

- We tested several classifiers, including Logistic Regression, Gradient Boosting, and Random Forest.
- **MLflow logged every experiment, tracking:**
 - **Parameters:** n_estimators, max_depth, etc.
 - **Metrics:** Accuracy, Precision, Recall, F1-Score.
- **Outcome:** The MLflow UI allowed for easy comparison, clearly identifying the best-performing model.

```
PS F:\project\titanic> python .\src\eval.py
Evaluation for best_model_RandomForestClassifier
Accuracy: 0.9529
Precision: 0.9491
Recall: 0.9269
F1: 0.9379
```

```
PS F:\project\titanic> python .\src\train.py
LogisticRegression Metrics: {'accuracy': 0.8100558659217877, 'precision': 0.7857142857142857, 'recall': 0.7432432432432432, 'f1': 0.7638888888888888}
RandomForestClassifier Metrics: {'accuracy': 0.8268156424581006, 'precision': 0.7945205479452054, 'recall': 0.7837837837837838, 'f1': 0.7891156462585034}
GradientBoostingClassifier Metrics: {'accuracy': 0.8268156424581006, 'precision': 0.8028169014084507, 'recall': 0.7702702702702703, 'f1': 0.7862068965517242}
Best model (RandomForestClassifier) saved to models/best_model_RandomForestClassifier.pkl with accuracy: 0.8268156424581006
```


Step 3: Serving the Model with API & Docker



Turning the Best Model into a Robust Service

- FastAPI Service:
 - Provides a high-performance web API.
 - GET /health: For health checks.
 - POST /predict: Accepts passenger data (JSON) and returns a survival prediction.
- Pydantic Validation:
 - Enforces the correct input data types and structure, automatically rejecting invalid requests.
- Docker Containerization:
 - Packages the API, model, and all dependencies into a single, portable container.
 - Benefit: Runs anywhere, consistently. "It works on my machine" becomes "It works everywhere."

Step 4: Full Automation with CI/CD

GitHub Actions: The Automated Assembly Line

- A workflow is triggered on every git push to the main branch.

The Automated Pipeline:

- **Lint & Test:** Automatically checks code quality and runs unit tests.
- **Build Docker Image:** Builds the application into a versioned Docker container.
- **Push to Registry:** Pushes the new image to Docker Hub.
- **Deploy :** Deploys the new container to the production environment.

Result: Fast, reliable, and hands-off deployments.

CI/CD workflow snapshot:

Triggered via push 12 hours ago

 **BreadKrum** pushed [7ab8cb3](#) [BreadKrum-patch-1](#)

Status

Success

Total duration

2m 50s

Artifacts

—

ci.yml
on: push

✓ build

2m 47s

✓ deploy

30s

CI/CD workflow snapshot:

← titanic

✓ Update ci.yml #7

🏠 Summary

Jobs

- ✓ build
- ✓ deploy

Run details

- 🕒 Usage
- 📄 Workflow file

deploy
succeeded 4 minutes ago in 30s

- > ✓ Set up job
- > ✓ Pull Docker image
- > ✓ restart container
- > ✓ Complete job

MLflow comparison

Experiments >

Titanic_Survival_Experiments

[Provide Feedback](#) [Add Description](#)

Runs Models **Experimental** Evaluation [Traces](#)

[Group by](#) [metrics.rmse < 1 and params.model = "tree"](#) [Time created](#) [State: Active](#) [Datasets](#) [Sort: accuracy](#) [Columns](#)

	Run Name	Created	Dataset	Duration	Source	Models	Metrics
☐	● RandomForest	✓ 17 minutes ago	-	6.3s	train.py	RandomForest_model	0.981741573...
☐	● RandomForest	✓ 20 minutes ago	-	6.5s	train.py	model	0.981741573...
☐	● GradientBoosting	✓ 17 minutes ago	-	6.4s	train.py	GradientBoosting_model	0.910112359...
☐	● GradientBoosting	✓ 20 minutes ago	-	6.7s	train.py	model	0.910112359...
☐	● LogisticRegression	✓ 17 minutes ago	-	8.6s	train.py	LogisticRegression_model	0.807584269...
☐	● LogisticRegression	✓ 20 minutes ago	-	20.7s	train.py	model	0.807584269...

Key Challenges in MLOps

This project directly addresses the most common MLOps pain points:

Solution Implemented

Reproducibility

- "It worked on my machine." Different data or library versions lead to different results.
- Git + DVC for versioning all code and data artifacts.

Automation

- Manual handoffs for testing and deployment are slow, tedious, and prone to human error.
- GitHub Actions (CI/CD) to automate the entire workflow from code push to deployment.

Deployment

- How do you package and serve a model so other applications can use it consistently?
- FastAPI + Docker to create a robust, containerized, and scalable API service.

Monitoring

- Once deployed, how do we know the model is still performing well and the service is healthy?
- Structured Logging & Monitoring Hooks to track API health, latency, and prediction distributions.

Model Performance

- Through rigorous experiment tracking, the Random Forest model was identified as the best-performing algorithm for this task.
- The final deployed model achieved a classification accuracy of 84.2% on the holdout test data.

Conclusion

Project Achievements:

- Successfully designed and built a complete, end-to-end MLOps pipeline.
- Established full reproducibility by versioning code (Git) and.
- Automated the entire lifecycle from testing to deployment using CI/CD with GitHub Actions.
- The system is robust, scalable, and maintainable.

Thank You!